

Architecture Intelligence Platform

Proof of Concept - Technical Specification

Python reference implementation

OpenAPI + AsyncAPI (Queues) -> Canonical Model -> Neo4j -> Architecture Analysis -> LLM Query

Version	1.0
Status	PoC Specification
Reference stack	Python 3.13, FastAPI, Pydantic, Neo4j
Communication	Synchronous REST + asynchronous queues
Date	25.08.2026

Inhalt

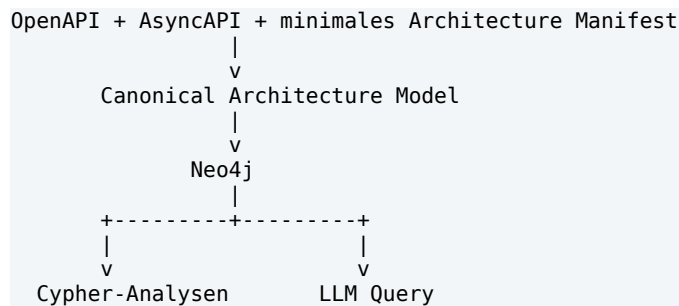
1. Ziel und PoC-Grenzen
2. Systemkontext und Zielarchitektur
3. Technologiestack
4. Canonical Architecture Model
5. Source und Ingestion Layer
6. OpenAPI Adapter
7. AsyncAPI Queue Adapter
8. Architecture Manifest
9. Provenance und Evidence
10. Validation
11. Neo4j Graph Model
12. Graph Import und Reconciliation
13. Analysis Engine
14. REST API
15. LLM Query Subsystem
16. Minimal UI
17. Konfiguration und Betrieb
18. Logging und Observability
19. Security
20. Tests
21. Akzeptanzkriterien
22. Repository-Struktur
23. Implementierungsplan
24. Erweiterung nach dem PoC

1. Ziel und PoC-Grenzen

Der PoC soll nachweisen, dass aus vorhandenen OpenAPI- und AsyncAPI-Spezifikationen automatisch ein Architecture Knowledge Graph aufgebaut werden kann, der synchrone REST-Kommunikation und asynchrone Queue-Kommunikation in einem gemeinsamen Modell abbildet.

Der Graph ist die strukturierte Faktenbasis. Standardanalysen werden deterministisch mit Cypher ausgeführt. Ein LLM dient ausschliesslich als natuerlichsprachliche Query- und Erklaerungsschicht ueber dem Graphen.

1.1 Kernhypothese



1.2 Im Scope

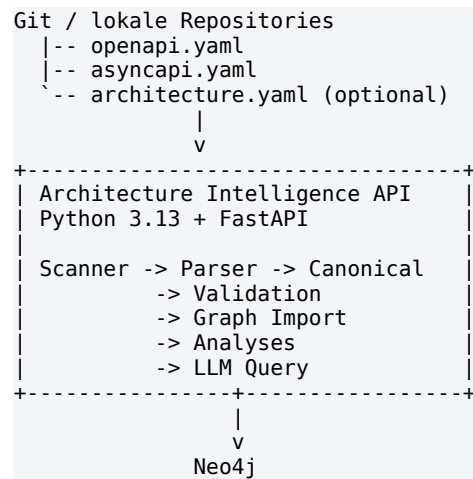
- Import mehrerer OpenAPI-Spezifikationen (OpenAPI 3.0/3.1).
- Import mehrerer AsyncAPI-Spezifikationen mit Queue-basierter Kommunikation.
- Abbildung von REST-Providern und -Callern.
- Abbildung von Queue-Sendern, Queue-Consumern, Messages, Schemas und DLQ-Beziehungen.
- Canonical Model als technologieunabhaengige Zwischenschicht.
- Persistenz und Abfrage in Neo4j.
- Fuenf definierte Architektur-Analysen.
- Read-only LLM Query: natuerliche Sprache -> validiertes Cypher -> Graphresultat -> Erklaerung.
- Minimale REST API und optionale kleine Web UI.
- Provenance fuer alle wesentlichen Architektur-Fakten.

1.3 Bewusst nicht im ersten PoC

- Kubernetes-/Cloud-Discovery.
- OpenTelemetry und Runtime-Traces.
- Vector Database und Dokumenten-RAG.
- ADR-/Ticket-/Quellcodeanalyse.
- Automatisch generiertes LLM-Wiki.
- Team Ownership und Organisationsgraph.
- CI/CD Policy Gates.
- Vollstaendige Promise-Theory- oder Semantic-Spacetime-Implementierung.

2. Systemkontext und Zielarchitektur

Der PoC wird als modularer Python-Monolith implementiert. Alle fachlichen Subkomponenten laufen in einem FastAPI-Prozess; Neo4j ist der einzige externe persistente Infrastrukturbaustein.



2.1 Kommunikationsmodell

Kommunikation	Canonical Flow	Quelle
Synchron / REST	Caller Service -> CALLS -> Operation <- PROVIDES <- Provider Service	OpenAPI + Architecture Manifest
Asynchron / Queue	Sender Service -> SENDS -> Queue <- RECEIVES_FROM <- Consumer Service	AsyncAPI
Payload	Queue -> CARRIES -> Message -> CONFORMS_TO -> Schema	AsyncAPI
REST Payload	Operation -> REQUEST_SCHEMA / RESPONSE_SCHEMA -> Schema	OpenAPI

3. Technologiestack

Bereich	Technologie	Begründung
Runtime	Python 3.13	Schnelle Iteration, starkes Parsing-/Graph-/LLM-Oekosystem.
Web/API	FastAPI	Typisierte API, OpenAPI-Generierung, geringe PoC-Komplexität.
Modelle	Pydantic v2	Validierung und Serialisierung des Canonical Models.
YAML/JSON	PyYAML / stdlib json	Direkte Verarbeitung von OpenAPI/AsyncAPI/Manifest.
Schema Validation	jsonschema	Validierung von JSON-Schema-basierten Artefakten.
Graph DB	Neo4j	Property Graph, Cypher, Visualisierung, Traversal/Impact Analysis.
Graph Driver	neo4j Python Driver	Offizieller Treiber; explizite Transaktionen.
Tests	pytest + Testcontainers	Unit- und Neo4j-Integrationstests.
Packaging	pyproject.toml	Modernes Dependency- und Build-

Bereich	Technologie	Begründung
		Setup.
Local runtime	Docker Compose	Einfacher lokaler Start von App + Neo4j.
Optional Analyse	NetworkX	Spätere experimentelle Graphmetriken; nicht MVP-kritisch.

4. Canonical Architecture Model

OpenAPI und AsyncAPI werden niemals direkt in das Neo4j-Schema geschrieben. Jeder Source Adapter erzeugt zuerst ein gemeinsames Canonical Model. Dadurch bleiben Parser, Graphpersistenz und spätere Datenquellen voneinander entkoppelt.

4.1 Kernentitäten

Entität	Pflichtfelder	Bedeutung
Service	id, name	Logischer Microservice.
Operation	id, service_id, method, path	REST-Operation.
Queue	id, name	Asynchroner Transport-/Puffer-Endpunkt.
Message	id, name	Semantischer Nachrichtentyp.
Schema	id, name, format	Payload- oder API-Datenschema.
Relation	type, source_id, target_id	Typisierte Architekturbeziehung.
Provenance	source_type, source_file, revision	Herkunft eines Fakts.

4.2 Pydantic Referenzmodell

```

from enum import StrEnum
from pydantic import BaseModel, Field

class Direction(StrEnum):
    SEND = "SEND"
    RECEIVE = "RECEIVE"

class Service(BaseModel):
    id: str
    name: str
    version: str | None = None

class Operation(BaseModel):
    id: str
    service_id: str
    operation_id: str | None = None
    method: str
    path: str
    request_schema_ids: list[str] = Field(default_factory=list)
    response_schema_ids: list[str] = Field(default_factory=list)

class Queue(BaseModel):
    id: str
    name: str
    protocol: str | None = None
    namespace: str | None = None
    queue_type: str = "STANDARD"

class Message(BaseModel):
    id: str
    name: str
    version: str | None = None
    schema_id: str | None = None

class Schema(BaseModel):
    id: str
    name: str
    version: str | None = None
    format: str | None = None
    canonical_hash: str | None = None

class ArchitectureModel(BaseModel):
    services: list[Service] = Field(default_factory=list)
    operations: list[Operation] = Field(default_factory=list)
    queues: list[Queue] = Field(default_factory=list)
    messages: list[Message] = Field(default_factory=list)
    schemas: list[Schema] = Field(default_factory=list)
    relations: list["Relation"] = Field(default_factory=list)
    provenance: list["Provenance"] = Field(default_factory=list)

```

4.3 Stabile IDs

Typ	Beispiel
Service	service:order-service
Operation	operation:product-service:GET:/products/{id}
Queue	queue:asb:commerce:payment-q
Message	message:PaymentRequested:v2
Schema	schema:PaymentRequested:v2

IDs dürfen nicht vom lokalen Repository-Pfad abhängen. Sie müssen über wiederholte Imports stabil sein und mehrere Repositories konfliktfrei zusammenführen können.

5. Source und Ingestion Layer

5.1 Specification Scanner

Der Scanner findet relevante Dateien in konfigurierten Verzeichnissen oder ueber explizite Source-Definitionen.

```
openapi.yaml | openapi.yml | openapi.json
asyncapi.yaml | asyncapi.yml | asyncapi.json
architecture.yaml

class SpecificationSource(BaseModel):
    path: Path
    type: SpecificationType
    service_id: str
    revision: str | None = None
```

5.2 Ingestion Pipeline

```
scan
-> parse
-> source-level validate
-> map to Canonical Model
-> canonical validate
-> reconcile / diff
-> transactional graph write
```

Fehler vor dem Graph-Write duerfen keinen partiellen Import hinterlassen. Ein Import eines Service-Artefakts ist entweder vollstaendig erfolgreich oder wird verworfen.

6. OpenAPI Adapter

Der OpenAPI Adapter extrahiert angebotene REST-Capabilities. Fuer den PoC wird die Provider-Seite automatisch aus der Spezifikation gewonnen; die Caller-Seite wird ueber das Architecture Manifest ergaenzt.

6.1 Zu extrahierende Informationen

- Service-Metadaten.
- HTTP-Methode und Pfad.
- operationId und Summary.
- Request Body Schema.
- Response Schemas je Statuscode.
- Wiederverwendete Component Schemas.
- Optional Security-Metadaten als Properties; keine tiefe Security-Analyse im MVP.

6.2 Mapping

```
ProductService
|
PROVIDES
|
v
GET /products/{id}
|
+-- REQUEST_SCHEMA --> ProductId
`-- RESPONSE_SCHEMA -> Product
```

6.3 Beispiel

```
paths:
  /products/{id}:
    get:
      operationId: getProduct
      responses:
        "200":
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Product"
```

7. AsyncAPI Queue Adapter

Der AsyncAPI Adapter ist auf Queue-basierte Kommunikation zugeschnitten. Queue und Message bleiben getrennte Entitäten, weil Queue-Eigenschaften, DLQ-Verhalten und Message-Typen unterschiedliche Architekturinformationen repräsentieren.

7.1 Zielmodell



7.2 Zu extrahierende Informationen

- Queue-/Channel-Name und technische Namespace-Information, soweit vorhanden.
- Send-/Receive-Richtung der Operation.
- Message-Name und Version.
- Payload Schema und Referenzen.
- Protokoll/Binding-Metadaten, soweit vorhanden.
- DLQ-Zuordnung, falls in der Spezifikation oder in Erweiterungsmetadaten vorhanden.

7.3 Competing Consumers

Mehrere Laufzeitinstanzen desselben Consumer-Service werden im statischen PoC nicht als separate Service-Nodes modelliert. Das Canonical Model beschreibt logische Services. Pod-/Instanzinformationen gehören in eine spätere Runtime-Schicht.

7.4 DLQ

```
payment-q -[:DEAD_LETTERS_T0]-> payment-dlq
```

8. Architecture Manifest

OpenAPI beschreibt typischerweise, was ein Provider anbietet, aber nicht, welcher andere Service die Operation tatsächlich aufruft. Für den engen PoC wird diese Lücke über ein minimales architecture.yaml geschlossen.


```

service: order-service
calls:
  - service: product-service
    operationId: getProduct
  - service: customer-service
    operationId: getCustomer

```

Das Manifest darf nur Informationen enthalten, die nicht bereits verlaesslich aus OpenAPI oder AsyncAPI ableitbar sind. Eine spaetere Version kann CALLS aus OpenTelemetry, Client-Code oder Service-Konfigurationen ableiten.

9. Provenance und Evidence

Provenance wird bereits im PoC als Pflichtkonzept eingefuehrt. Das LLM darf nur Aussagen formulieren, deren zugrunde liegende Graph-Fakten eine nachvollziehbare Quelle besitzen.

```

class Provenance(BaseModel):
    source_type: str # OPENAPI | ASYNCAPI | MANIFEST
    source_file: str
    source_revision: str | None = None
    evidence_type: str = "DECLARED"

```

Evidence Type	Bedeutung	PoC
DECLARED	Aus Spezifikation/Manifest abgeleitet.	Ja
OBSERVED	Zur Laufzeit beobachtet, z.B. OpenTelemetry.	Spaeter
INFERRED	Aus Dokumenten/LLM/Regeln hergeleitet.	Spaeter

10. Validation

Die Validation ist in Source-Validation und Canonical-Validation getrennt. Source-Validation prueft Syntax und Referenzen des Eingabeartefakts; Canonical-Validation prueft Architekturregeln des normalisierten Modells.

ID	Regel
V1	Jeder Service besitzt eine eindeutige stabile ID.
V2	Jede REST-Operation gehoert genau einem Provider-Service.
V3	Jede Queue besitzt eine eindeutige technische ID.
V4	Jede Message besitzt eine eindeutige ID.
V5	Jeder CALLS-Eintrag referenziert eine existierende Operation.
V6	Eine Schema-Referenz verweist auf ein existierendes Schema.
V7	Eine DLQ darf nicht auf sich selbst zeigen.
V8	Relationen referenzieren nur existierende Source-/Target-Entities.
V9	Ein Service-Import ist atomar: bei Fehler kein Teilupdate.

11. Neo4j Graph Model

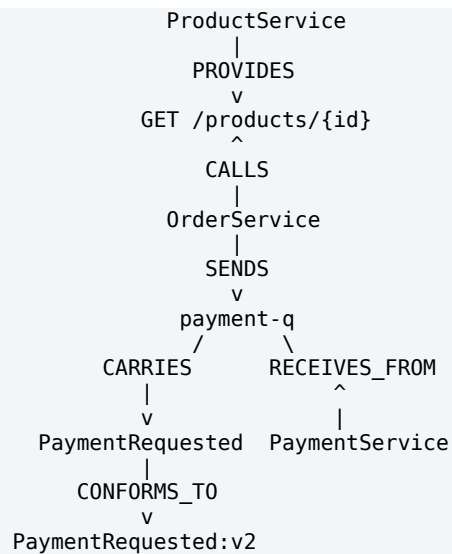
11.1 Node Labels

Service
Operation
Queue
Message
Schema

11.2 Relations

Relation	Von -> Nach	Bedeutung
PROVIDES	Service -> Operation	REST-Provider.
CALLS	Service -> Operation	REST-Caller.
REQUEST_SCHEMA	Operation -> Schema	Request Payload.
RESPONSE_SCHEMA	Operation -> Schema	Response Payload.
SENDS	Service -> Queue	Asynchroner Sender.
RECEIVES_FROM	Service -> Queue	Asynchroner Consumer.
CARRIES	Queue -> Message	Message-Typ auf Queue.
CONFORMS_TO	Message -> Schema	Message Payload Schema.
DEAD_LETTERS_TO	Queue -> Queue	DLQ-Beziehung.

11.3 Beispielgraph



11.4 Constraints und Indizes

```
CREATE CONSTRAINT service_id IF NOT EXISTS
FOR (s:Service) REQUIRE s.id IS UNIQUE;

CREATE CONSTRAINT operation_id IF NOT EXISTS
FOR (o:Operation) REQUIRE o.id IS UNIQUE;

CREATE CONSTRAINT queue_id IF NOT EXISTS
FOR (q:Queue) REQUIRE q.id IS UNIQUE;

CREATE CONSTRAINT message_id IF NOT EXISTS
FOR (m:Message) REQUIRE m.id IS UNIQUE;

CREATE CONSTRAINT schema_id IF NOT EXISTS
FOR (s:Schema) REQUIRE s.id IS UNIQUE;
```

12. Graph Import und Reconciliation

Der Graph Importer schreibt das validierte Canonical Model transaktional in Neo4j. Persistente Entities werden mit MERGE behandelt. Der Import muss idempotent sein.

12.1 Referenzablauf

```
Canonical Model
  |
  v
Reconciliation / Diff
  |
  v
Neo4j Transaction
  |
  +-- MERGE nodes
  +-- MERGE current relations
  `-- remove/expire stale facts for imported service
```

12.2 PoC-Reimport-Strategie

Fuer den ersten PoC ist ein serviceweiser Full-Reimport akzeptabel: alle DECLARED-Fakten, deren Provenance zu diesem Service gehoert, werden in einer Transaktion ersetzt. Global geteilte Entities wie Queues, Messages und Schemas werden ueber stabile IDs zusammengefuehrt und nur entfernt, wenn keine Provenance mehr auf sie verweist.

13. Analysis Engine

Die Standardanalysen sind deterministisch und benoetigen kein LLM. Sie werden als fest definierte, parametrisierte Cypher-Queries implementiert.

13.1 A1 - Sender einer Queue

```
MATCH (s:Service)-[:SENDS]->(q:Queue {id:$queue_id})
RETURN s.id, s.name
ORDER BY s.name
```

13.2 A2 - Consumer einer Queue

```
MATCH (s:Service)-[:RECEIVES_FROM]->(q:Queue {id:$queue_id})
RETURN s.id, s.name
ORDER BY s.name
```

13.3 A3 - Queues mit Sender aber ohne Consumer

```
MATCH (q:Queue)
WHERE EXISTS { MATCH (:Service)-[:SENDS]->(q) }
AND NOT EXISTS { MATCH (:Service)-[:RECEIVES_FROM]->(q) }
RETURN q.id, q.name
ORDER BY q.name
```

13.4 A4 - Consumer-Queues ohne bekannten Sender

```
MATCH (consumer:Service)-[:RECEIVES_FROM]->(q:Queue)
WHERE NOT EXISTS { MATCH (:Service)-[:SENDS]->(q) }
RETURN consumer.name, q.id, q.name
ORDER BY q.name, consumer.name
```

13.5 A5 - Mixed Architecture Blast Radius

Der Blast Radius folgt sowohl synchronen REST-Abhängigkeiten als auch asynchronen Queue-Flows. Direkte Service-Service-Kanten werden aus den Primaerfakten abgeleitet; die Primaerfakten bleiben erhalten.

```
SYNC:
A -[:CALLS]-> Operation <-[:PROVIDES]- B

ASYNC:
A -[:SENDS]-> Queue <-[:RECEIVES_FROM]- B

Traversal:
Service -> {SYNC | ASYNC} -> Service -> ...
maxDepth = 5 (configurable)
```

13.6 Abgeleitete Relations

SYNC_DEPENDS_ON und ASYNC_FLOW_TO koennen fuer schnellere Visualisierung materialisiert werden, sind im PoC aber besser als berechnete Sicht zu behandeln. So bleibt der Graph frei von redundanten Wahrheiten.

14. REST API des PoC

Methode	Endpoint	Zweck
GET	/api/services	Services auflisten.
GET	/api/services/{serviceld}	Service-Detail.
GET	/api/queues	Queues auflisten.
GET	/api/queues/{queueId}	Queue-Detail.
GET	/api/messages	Messages auflisten.
GET	/api/messages/{messageld}	Message-Detail.
GET	/api/analysis/queues/{queueId}/senders	A1.
GET	/api/analysis/queues/{queueId}/consumers	A2.
GET	/api/analysis/queues/without-consumers	A3.
GET	/api/analysis/queues/without-senders	A4.
GET	/api/analysis/services/{serviceld}/blast-radius	A5.
POST	/api/import	Import aller konfigurierten Sources.
POST	/api/import/service/{serviceld}	Service-Reimport.
POST	/api/query	Natuerlichsprachliche Graphfrage.

14.1 FastAPI Skeleton

```
from fastapi import FastAPI

app = FastAPI(title="Architecture Intelligence PoC")

@app.get("/api/services/{service_id}")
def get_service(service_id: str):
    return service_query.get_service(service_id)

@app.get("/api/analysis/services/{service_id}/blast-radius")
def blast_radius(service_id: str, depth: int = 5):
    return analysis_service.blast_radius(service_id, depth)
```

15. LLM Query Subsystem

Der LLM-Teil hat eine einzige Funktion: natuerliche Sprache auf sichere Graphabfragen abbilden und das Ergebnis erklären. Der Graph bleibt die Faktenquelle; das LLM ist nicht die Knowledge Base.

```
Question
  |
  v
Intent / Cypher Generation
  |
  v
Cypher Validator
  |
  v
Neo4j read-only
  |
  v
Rows + Provenance
  |
  v
LLM Explanation
```

15.1 Komponenten

Komponente	Verantwortung
ArchitectureQuestionService	Orchestriert Frage, Query, Execution und Erklarung.
CypherGenerator	Generiert Cypher aus Frage + festem Graphschema.
CypherValidator	Prueft Allowlist, Depth und Result-Limits.
ReadOnlyGraphExecutor	Fuehrt Query ueber read-only Neo4j User aus.
AnswerComposer	Formuliert Antwort ausschliesslich aus Result Rows und Provenance.

15.2 Erlaubte Cypher-Konstrukte

```
MATCH
OPTIONAL MATCH
WHERE
WITH
RETURN
ORDER BY
LIMIT
```

15.3 Verbotene Konstrukte

```
CREATE
DELETE
DETACH DELETE
SET
REMOVE
MERGE
DROP
LOAD CSV
CALL
```

15.4 Zusätzliche Limits

- Neo4j Benutzer mit reinem Lesezugriff.
- Maximale Traversal-Tiefe: standardmaessig 5.
- Maximale Ergebniszeilen: standardmaessig 100.
- Nur erlaubte Node Labels und Relation Types.
- Im PoC Anzeige des generierten Cypher-Statements zur Nachvollziehbarkeit.
- Kein direkter Tool-Zugriff des LLM auf Neo4j Credentials.

15.5 LLM Provider Abstraction

Die Implementierung soll einen Provider Adapter verwenden, damit das PoC nicht an einen einzelnen Modellanbieter gekoppelt ist. Das Interface erwartet strukturierte Outputs fuer Cypher und Answer Composition; konkrete Provider-Konfiguration gehoert in Environment/Config.

16. Minimal UI

Die UI ist nicht Kern des PoC, sollte aber drei Nutzungsfaelle sichtbar machen. Sie kann als kleine React-Anwendung oder als sehr einfache serverseitige/HTML-Oberflaeche realisiert werden.

16.1 Service Explorer

```
OrderService

Provides REST
  POST /orders
  GET /orders/{id}

Calls REST
  ProductService / getProduct

Sends to queues
  payment-q

Receives from queues
  payment-result-q

Downstream
  ProductService (sync)
  PaymentService (async)
```

16.2 Queue Explorer

```
payment-q

Protocol: AMQP
Senders: OrderService
Consumers: PaymentService
Messages: PaymentRequested:v2
DLQ: payment-dlq
```

16.3 Natural Language Query

Die Query-Seite zeigt Benutzerfrage, generiertes Cypher, Result Rows und die erklarte Antwort. Diese Transparenz ist im PoC wichtiger als eine aufwendige UI.

17. Konfiguration und Betrieb

17.1 Beispielkonfiguration

```
architecture_intelligence:
  sources:
    directories:
      - ./repositories

  graph:
    uri: bolt://neo4j:7687
    database: neo4j
    max_traversal_depth: 5

  import:
    openapi: true
    asyncapi: true
    architecture_manifest: true

  llm:
    enabled: true
    max_result_rows: 100
```

17.2 Environment Secrets

```
NEO4J_USER
NEO4J_PASSWORD
LLM_API_KEY
```

Secrets dürfen nicht im Repository oder in Graph-Properties gespeichert werden.

17.3 Docker Compose

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - neo4j

  neo4j:
    image: neo4j
    ports:
      - "7474:7474"
      - "7687:7687"
```

18. Logging und Observability des PoC

Auch wenn OpenTelemetry selbst noch keine Architekturquelle ist, benötigt der PoC ausreichend technische Observability fuer Import- und Query-Fehler.

- Strukturiertes Logging mit service_id, source_file, import_id und duration_ms.
- Metriken: Importdauer, Anzahl Nodes/Relations pro Import, Validation Errors, LLM Query Count, Cypher Validation Rejects.
- Health Endpoints fuer App und Neo4j Connectivity.
- Keine Speicherung sensibler Prompt-/Credential-Daten in Logs.

```
Imported service=order-service
openapi_operations=12
asyncapi_queues=4
messages=6
relations=31
duration_ms=423
warnings=0
```

19. Security

Bereich	PoC-Anforderung
Neo4j	Separater read-write User fuer Import; separater read-only User fuer Analysen/LLM.
LLM	Kein direkter DB-Schreibzugriff; Cypher muss validiert werden.
Secrets	Nur Environment/Secret Store; nicht in Source oder Graph.
Files	Scanner nur auf konfigurierten Root-Verzeichnissen; kein beliebiger Path Traversal.
API	PoC lokal oder intern; produktive Authentifizierung ist spaetere Ausbaustufe.
Prompt Injection	LLM erhaelt Graphschema und Queryresultate, keine unkontrollierten Dokumente im MVP.

20. Tests

20.1 Unit Tests

- Canonical ID Generator.
- OpenAPI Adapter.
- AsyncAPI Queue Adapter.
- Architecture Manifest Adapter.
- Canonical Validation.
- Provenance Mapping.
- Cypher Validator.
- Blast Radius Traversal Logic.

20.2 Integration Tests

Neo4j wird mit Testcontainers gestartet. Ein Test importiert Spezifikationen, fuehrt Cypher aus und prueft Nodes, Relations und Analysen.

```

Specification fixtures
  |
  v
Python Importer
  |
  v
Neo4j Testcontainer
  |
  v
Cypher assertions

```

20.3 Testlandschaft

Service	REST	Queue
OrderService	calls ProductService	sends payment-q
ProductService	provides getProduct	-
PaymentService	optional external REST call	receives payment-q; sends invoice-q
InvoiceService	-	receives invoice-q

Zusaetzliche Fixtures: unused-q mit Sender aber ohne Consumer sowie unknown-producer-q mit Consumer aber ohne bekannten Sender.

21. Akzeptanzkriterien

- AC1: OpenAPI-Dateien mehrerer Services werden reproduzierbar importiert.
- AC2: AsyncAPI-Dateien mit Queue-Kommunikation werden reproduzierbar importiert.
- AC3: REST Provider werden korrekt erkannt.
- AC4: REST Caller werden ueber das Architecture Manifest korrekt verbunden.
- AC5: Queue Sender und Consumer werden korrekt erkannt.
- AC6: Queue, Message und Schema werden als getrennte Entities korrekt verknuepft.
- AC7: DLQ-Beziehungen werden dargestellt.
- AC8: Wiederholte Imports erzeugen keine Duplikate.
- AC9: Die fuenf Standardanalysen liefern deterministische Ergebnisse.
- AC10: Der Blast Radius kombiniert synchrone und asynchrone Pfade.
- AC11: Eine natuerlichsprachliche Frage wird in eine sichere read-only Cypher-Abfrage uebersetzt.
- AC12: Das LLM kann Neo4j nicht veraendern.
- AC13: Wesentliche Beziehungen besitzen nachvollziehbare Provenance.
- AC14: Ein fehlerhafter Import erzeugt keinen inkonsistenten Teilzustand.
- AC15: Ein Entwickler kann Service- und Queue-Zusammenhaenge ohne manuelle Repository-Suche nachvollziehen.

22. Repository-Struktur

```

architecture-intelligence-poc/
|-- pyproject.toml
|-- Dockerfile
|-- docker-compose.yml
|-- README.md
|-- app/
|   |-- main.py
|   |-- settings.py
|   |-- canonical/
|   |   |-- model.py
|   |   |-- ids.py
|   |-- ingestion/
|   |   |-- scanner.py
|   |   |-- openapi_adapter.py
|   |   |-- asyncapi_adapter.py
|   |   |-- manifest_adapter.py
|   |-- provenance/
|   |   |-- model.py
|   |-- validation/
|   |   |-- source_validation.py
|   |   |-- canonical_validation.py
|   |-- graph/
|   |   |-- repository.py
|   |   |-- importer.py
|   |   |-- reconciliation.py
|   |   |-- schema.py
|   |-- analysis/
|   |   |-- queues.py
|   |   |-- dependencies.py
|   |   |-- blast_radius.py
|   |-- ai/
|   |   |-- question_service.py
|   |   |-- provider.py
|   |   |-- cypher_generator.py
|   |   |-- cypher_validator.py
|   |   |-- answer_composer.py
|   |-- api/
|   |   |-- services.py
|   |   |-- queues.py
|   |   |-- messages.py
|   |   |-- analysis.py
|   |   |-- import_api.py
|   |   |-- query.py
|-- tests/
|   |-- unit/
|   |-- integration/
|   |-- fixtures/
|-- examples/
|   |-- order-service/
|   |-- product-service/
|   |-- payment-service/
|   |-- invoice-service/

```

23. Implementierungsplan

Iteration	Inhalt	Exit-Kriterium
1	Canonical Model + Stable IDs + Fixtures	Modelle validieren Beispielarchitektur.
2	OpenAPI Adapter	REST Operations/Schemas werden korrekt erzeugt.
3	AsyncAPI Queue Adapter	Queues/Messages/Sender/Consumer werden korrekt erzeugt.
4	Neo4j Schema + Importer	Idempotenter Graphimport.

Iteration	Inhalt	Exit-Kriterium
5	Architecture Manifest	REST Caller sind im Graph verbunden.
6	5 Standardanalysen	Alle Cypher-Analysen bestehen Integrationstests.
7	FastAPI + minimale UI	Service/Queue/Analyse sichtbar.
8	LLM Query + Validator	Natuerliche Frage -> read-only Cypher -> Antwort.
9	PoC Review	Mehrwert anhand realer Spezifikationen bewertet.

23.1 PoC-Erfolgsmessung

1. Die Plattform beantwortet Architekturfragen schneller als manuelle Repository-Recherche.
2. Der Graph entdeckt mindestens einige nicht offensichtliche Queue-/Service-Abhaengigkeiten oder Dokumentationsluecken.
3. Die fuenf Standardanalysen sind ohne LLM reproduzierbar.
4. Das LLM verbessert Bedienbarkeit, ohne die Faktenquelle zu ersetzen.
5. Das Canonical Model kann spaeter OpenTelemetry als neue Quelle aufnehmen, ohne OpenAPI-/AsyncAPI-Adapter umzubauen.

24. Erweiterung nach dem PoC

Die erste empfohlene Erweiterung ist OpenTelemetry. Damit kann die deklarierte Architektur aus OpenAPI/AsyncAPI mit der beobachteten Runtime-Architektur verglichen werden.

```
DECLARED
OpenAPI + AsyncAPI + Manifest

vs.

OBSERVED
OpenTelemetry
```

24.1 Neue Analysen

- Observed - Declared: reale, aber nicht dokumentierte Abhaengigkeiten.
- Declared - Observed: deklarierte, aber eventuell veraltete oder ungenutzte Abhaengigkeiten.
- Kausale End-to-End-Flows ueber REST und Queues.
- Runtime-basierter Blast Radius.
- Vergleich logischer und physischer Locality nach spaeterer Kubernetes-/Cloud-Integration.

24.2 Spaetere Architecture Intelligence Platform

```
Static Architecture Graph
+
Observed Runtime Graph
+
Document / Vector Knowledge
+
LLM Query and Wiki
=
Architecture Intelligence Platform
```

Promise Theory und Semantic Spacetime koennen spaeter als semantische Interpretationsschicht ueber dem technisch etablierten Graphmodell untersucht werden. Fuer den PoC bleiben sie bewusst ausserhalb des implementierten Kernmodells.

Anhang A - Abhaengigkeitssemantik

A.1 Synchrone Abhaengigkeit

```
A -[:CALLS]-> O <-[:PROVIDES]- B
=> A synchronously depends on B through operation O
```

A.2 Asynchroner Flow

```
A -[:SENDS]-> Q <-[:RECEIVES_FROM]- B
=> message flow A -> Q -> B
```

Bei mehreren Instanzen von B beschreibt der Graph weiterhin nur den logischen Consumer-Service B. Competing-Consumer-Instanzen werden erst in einer Runtime-/Deployment-Erweiterung modelliert.

A.3 Queue und Message

```
Q -[:CARRIES]-> M -[:CONFORMS_TO]-> S
```

Die Trennung ist Pflicht, damit Transportparameter der Queue und semantische Payload-Eigenschaften der Message unabhaengig analysiert und versioniert werden koennen.

Anhang B - Designentscheidungen

Entscheidung	Begrueundung
Python statt Java	PoC-Schwerpunkt liegt auf Parsing, Transformation, Graphanalyse und LLM-Integration.
FastAPI statt Microservice-Splitting	Ein modularer Monolith reduziert Betriebs- und Integrationsaufwand.
Pydantic Canonical Model	Klare Validierung und Technologieentkopplung.
Neo4j Property Graph	Direkte Modellierung typisierter Beziehungen und Cypher-Analysen.
Queue als eigene Entity	Pufferung, DLQ, Competing Consumers und Queue-spezifische Eigenschaften sind architekturelevant.
Message getrennt von Queue	Semantik/Schema und Transport werden nicht vermischt.
Architecture Manifest fuer REST Caller	OpenAPI allein kennt in der Regel nur Provider, nicht Caller.
LLM nur read-only	Fakten bleiben deterministisch und auditierbar.
Provenance ab Tag 1	Spaetere LLM-Antworten muessen auf Quellen zurueckfuehrbar sein.
OpenTelemetry erst nach PoC	Zuerst statischen Mehrwert beweisen, danach DECLARED vs OBSERVED.

Ende der Specification